

LeWM Billiards Planning — Research Findings

Overview

We trained a JEPA-style world model on billiards expert demonstrations and evaluated whether learned embeddings can serve as a planning signal via Cross-Entropy Method (CEM) Model Predictive Control (MPC).

Four approaches were tested across three planning paradigms; only the state-based planner succeeded.

The probe experiment provides the sharpest diagnostic: the embedding *does* contain accurate ball position ($R^2=0.988$), but the dynamics predictor cannot simulate the collision mechanics needed to pot a ball — so no cost function can rescue CEM.

Experiments

Evaluation Setup

Task: Pot the target ball into any pocket.

Two evaluation combos:

- **Combo A** — Same-episode: ep 500 frame 0 → ep 500 last frame (goal)
- **Combo B** — Cross-episode: ep 100 frame 0 → ep 3000 last frame (goal)

Dataset: `billiards_expert_train.h5` — 4000 episodes, 971,321 frames, 96×96×3 pixels, action range $\approx [-30.4, +30.0]$, mean ≈ 0 , std ≈ 1 .

Experiment 1 — Embedding-based CEM, Original Model

Model: ViT-tiny encoder, `embed_dim=192`, `sigreg_weight=0.09`, trained 20 epochs (best checkpoint: `lewm_epoch_19_object.ckpt`, `val/pred_loss = 0.00737`).

Planner: CEM MPC, `PLAN_HORIZON=10`, `NUM_SAMPLES=1000`, `N_CEM_ITERS=15`, `TOPK=100`, `CEM_CLAMP=30.0`.

Cost function: MSE between predicted final embedding and goal embedding.

Results:

Combo	emb_dist_to_goal (range)	Final cost	Outcome
A	18–22 (flat)	290	FAILURE
B	18–22 (flat)	203	FAILURE

Key diagnostic metrics:

- Pairwise embedding L2: mean = 19.38 ± 1.56 , CV = 0.081

- Min pairwise L2 = 10.16 (no clustering of similar frames)
- Prediction error (pred→real L2): 0.946

The embedding distance to goal never decreased across 150 steps — CEM received no gradient signal.

Experiment 2 — Embedding-based CEM, Small Model (Architectural Fix)

Model: Same ViT-tiny architecture, `embed_dim=32`, `sigreg_weight=0.01`, trained 10 epochs (best checkpoint: `lewm_small_epoch_8_object.ckpt`, `val/pred_loss = 0.00284`).

Planner: CEM MPC, `PLAN_HORIZON=10`, `NUM_SAMPLES=1000`, `N_CEM_ITERS=20`, `TOPK=50`, `CEM_CLAMP=3.0` (corrected for `std≈1` actions).

Cost function: Same — MSE between predicted final embedding and goal embedding.

Results:

Combo	emb_dist_to_goal (range)	Final cost	Outcome
A	8.2–8.3 (flat)	~70	FAILURE
B	9.3–9.7 (flat → locked at 9.737)	~94	FAILURE

Key diagnostic metrics (vs original model):

- Pairwise embedding L2: CV = 0.121 (vs 0.081) — less uniform
- Min pairwise L2 = 0.89 (vs 10.16) — similar frames now cluster
- Prediction error (pred→real L2): 0.111 (vs 0.946) — **8.5× better**

The architectural fix (smaller `embed_dim` + weaker sigreg regularisation) significantly improved prediction accuracy and embedding structure. However, the CEM cost still never approached zero, and `emb_dist_to_goal` remained flat throughout. For Combo B, the ball reached a stopped configuration from step 84 onward and the planner locked into a fixed cost of ~93.9.

Experiment 3 — State-based CEM (No World Model)

Planner: CEM runs rollouts directly in the billiards simulator (no neural network involved in cost computation). Cost = minimum distance from target ball to any pocket across the simulated trajectory.

Parameters: `PLAN_HORIZON=20`, `EXEC_PER_PLAN=5`, `MAX_EVAL_STEPS=300`, `NUM_SAMPLES=500`, `N_CEM_ITERS=20`, `TOPK=50`, `ACT_LO=-30.0`, `ACT_HI=30.0`.

Results:

Combo	Steps to success	Min dist to pocket	Outcome
A	9	0.02 px	SUCCESS
B	13	0.39 px	SUCCESS

This approach works because the cost function is directly informative about the task (ball position relative to pockets), and the simulator is fast enough for CEM rollouts without requiring a learned model.

Experiment 4 — Probe-based CEM (Linear Probe on Frozen Embeddings)

Probe architecture: 2-layer MLP, `embed_dim(32) → hidden(64) → state(10)`.

Training: 1000 random frames from the dataset, 500 epochs, Adam lr=1e-3.

Probe training results:

Epoch	MSE	Mean R ²
0	1.027	-0.003
100	0.392	0.611
200	0.362	0.641
300	0.340	0.661
400	0.327	0.676
499	0.314	0.687

Per-dimension R² (final probe on training data):

Dimension	R ²	Quality
cue_x	+0.854	good
cue_y	+0.731	moderate
cue_vx	+0.351	poor
cue_vy	+0.325	poor
tgt_x	+0.988	excellent
tgt_y	+0.976	excellent
tgt_vx	+0.366	poor
tgt_vy	+0.359	poor
pkt_x	+0.981	excellent
pkt_y	+0.937	excellent
mean	0.687	

Target ball position (the planning-relevant signal) is decoded with R²=0.99/0.98. The embedding *does* contain accurate ball position information.

CEM cost function: `||probe(predicted_emb_last) - goal_state_norm||2`

Planning results:

Combo	probe_dist (range)	Best cost	Outcome
A	2.73–2.79 (flat)	7.354	FAILURE
B	1.85–1.87 (flat)	1.573	FAILURE

probe_dist was completely flat in both combos — no action sequence moved the decoded predicted state closer to the goal.

Velocity Encoding Insight

The per-dimension R² breakdown reveals a systematic split between position and velocity:

Signal type	Dimensions	R ² range	Interpretation
Position	tgt_x, tgt_y	0.976–0.988	Excellent — model knows where the ball is
Position	pkt_x, pkt_y	0.937–0.981	Excellent — model knows pocket locations
Position	cue_x, cue_y	0.731–0.854	Good — cue position well encoded
Velocity	cue_vx, cue_vy	0.325–0.351	Poor — cue velocity barely encoded
Velocity	tgt_vx, tgt_vy	0.359–0.366	Poor — target velocity barely encoded

Why this matters for billiards specifically:

Potting a ball is a momentum-transfer problem. Success depends critically on:

1. The *velocity* of the cue at impact (determines transfer force)
2. The *direction* of the resulting target velocity (determines pocket angle)
3. The *speed* after collision (determines whether the ball reaches the pocket)

A world model that barely encodes velocity (R²≈0.35) cannot reliably predict whether a given cue shot will pot the target ball. Even if positions are known precisely, a collision event produces a discontinuous velocity change that the model must predict accurately to be useful for planning. The model fails at this.

Why single-frame embeddings can't encode velocity:

Velocity is inherently a temporal quantity — it cannot be read from a single frame without motion blur or explicit optical flow. JEPA encodes individual frames; the only velocity information available is implicit in the *pattern* of positions across the history context (3 frames), which is insufficient for precise momentum.

Comparison Table

Approach	Model used	Cost function	Combo A	Combo B	Steps
Embedding CEM (orig)	embed_dim=192	MSE in embedding space	FAILURE	FAILURE	150

Approach	Model used	Cost function	Combo A	Combo B	Steps
Embedding CEM (small)	embed_dim=32	MSE in embedding space	FAILURE	FAILURE	150
State-based CEM	None (simulator)	Dist to pocket	SUCCESS	SUCCESS	9 / 13
Probe-based CEM	embed_dim=32	MLP(emb)→state dist ²	FAILURE	FAILURE	150

Root Cause Analysis

Why embedding-based CEM fails

The fundamental problem is not prediction quality — it is cost landscape structure.

JEPA trains the predictor to minimise MSE between predicted and actual future embeddings. This objective does not require that:

1. L2 distance in embedding space $\propto$ semantic distance between scenes, or
2. Actions that move the ball toward a pocket also move predicted embeddings toward the goal embedding.

In other words, the embedding space may accurately predict *what the next frame looks like* without organising itself so that CEM can navigate from start to goal by following a cost gradient.

Evidence:

- The small model's `emb_dist_to_goal` stayed flat at ~8–10 throughout all 150 steps, even as the CEM searched over $1000 \times 20 = 20,000$ action sequences per step. The cost landscape is flat — no action sequence finds a path that decreases embedding distance to the goal.
- The original model's embedding space was more uniform (CV=0.081, min pairwise L2=10.16): literally no clustering of semantically similar frames.
- The small model improved this (CV=0.121, min pairwise L2=0.89) but not enough to provide a navigable gradient for CEM.

Why the architectural fix helped prediction but not planning

Reducing `embed_dim` (192→32) forces the encoder to discard redundant dimensions and represent only the most informative features. Reducing `sigreg_weight` (0.09→0.01) weakens the regularisation that pushes all embeddings toward a fixed Gaussian — allowing the embedding space to develop more structure.

These changes improved:

- Prediction accuracy 8.5×
- Minimum pairwise L2 from 10.16 to 0.89 (similar frames cluster)

But JEPA's training signal is "predict the next frame correctly," not "organise embeddings so that L2 distance reflects task-relevant state similarity." The necessary alignment between embedding geometry and task geometry was not learned.

Why the probe also failed (key finding)

The linear probe experiment provides the critical diagnosis.

What the probe confirmed:

- The embedding *does* contain ball position information: $R^2=0.988$ for `tgt_x`, $R^2=0.976$ for `tgt_y`, $R^2=0.981$ for nearest pocket x.
- Decoding state from a real frame's embedding works well.

What the probe revealed:

- `probe_dist` stayed flat throughout all 150 steps for both combos — identical to the raw embedding failure.
- The best CEM plan in Combo B achieved cost=1.57 (sqrt≈1.25 normalised units from goal state) and never approached 0.

The bottleneck is not the cost function — it is the **dynamics model's predicted embeddings**. When the predictor produces `predicted_emb` for a 10-step action sequence, those embeddings do not correspond to physically accurate future states of the billiards environment. Specifically:

- The predictor cannot simulate the contact mechanics needed to pot a ball. Potting requires a precise cue → target → pocket alignment; the predictor likely learned average-case dynamics rather than the sharp nonlinearity of collision.
- The minimum achievable CEM cost (~1.57) is a floor imposed by the model — no action sequence in the search space causes the predicted decoded state to reach a potted configuration.
- This is a **model exploitation / Dyna failure mode**: CEM finds sequences that look good under the model but don't correspond to physically achievable outcomes.

Conclusion: The world model is insufficiently accurate at predicting the consequence of actions over a 10-step horizon for CEM to exploit it. The state-based planner succeeds because it runs the *real* dynamics — no model error. Any JEPA-based planner will fail until the model can accurately predict the result of a potting shot.

What Would Be Needed to Fix Pure JEPA Planning

Option 1 — Linear probe on top of frozen embeddings × TRIED, FAILED

Trained a 2-layer MLP f : `embedding` → `state_vector` on 1000 labelled frames. Used $||f(\text{predicted_emb}) - \text{goal_state}||$ as the CEM cost.

- Result: FAILURE. `probe_dist` flat at ~1.85–2.78 for all 150 steps.
- The embedding *does* contain state information ($R^2=0.988$ for `tgt_x`), but the *predicted* embeddings from the dynamics model don't correspond to physically accurate future states. The model cannot imagine a trajectory that pots the ball.
- Root cause: dynamics model accuracy, not cost function design.

Option 2 — Frame stacking for implicit velocity encoding

The probe experiment showed velocity $R^2 \approx 0.35$ — the model barely knows ball speed or direction. The predictor receives 3 history frames, but the ViT encoder processes each independently. Stacking multiple frames as channels before encoding would give the encoder direct access to optical flow:

```
# Instead of: encoder(frame_t) → emb_t
# Do:         encoder(cat([frame_{t-2}, frame_{t-1}, frame_t])) → emb_t
```

This is cheap (no architecture change to the predictor) and should raise velocity R^2 significantly, which is necessary (though not sufficient) for planning to work.

Option 3 — Auxiliary velocity prediction head

Add a lightweight head on top of the encoder that explicitly predicts $[v_x, v_y]$ for both balls and train it jointly with the JEPA loss:

```
L_total = L_jepa + λ_vel · MSE(vel_head(emb), gt_velocity)
```

This forces the embedding to represent velocity explicitly, which the probe showed it currently lacks. The head can be discarded at inference — its only role is to shape the embedding during training.

Option 4 — Contrastive or VICReg objective

Replace or augment the JEPA prediction loss with a loss that explicitly aligns embedding distance with state distance:

```
L_total = L_jepa + λ · ||d_emb(s1,s2) - d_state(s1,s2)||²
```

This directly trains the embedding space to be metrically aligned with the task-relevant state, fixing the flat cost landscape.

Option 5 — Contrastive loss over goal-reachable pairs

A targeted variant: sample pairs of states where one can reach the other (from expert trajectories, they are adjacent in time or within the same episode) and pairs that cannot (different episodes). Train:

```
L_contrastive = push_apart(cross-episode pairs) + pull_together(same-episode pairs)
```

This would specifically align the embedding space with the notion of "reachability" rather than raw state similarity, which is exactly what CEM needs.

Option 6 — Goal-conditioned JEPA

Condition the predictor on a goal embedding and train with a reach reward. This requires adding a goal-conditioning head and modifying the training pipeline, but produces an embedding space explicitly structured for planning.

Option 7 — Hybrid cost function (pure accuracy fix)

Use the world model for dynamics rollouts (predicting future frames cheaply) but replace the CEM cost with the real simulator check on the decoded state:

```
cost = state_decoder(predicted_emb[-1]) → ball_position → dist_to_pocket
```

This would work only if the dynamics predictor becomes accurate enough to imagine a potting shot. The current model cannot — this requires either more data, more capacity, or better training objectives (Options 2–5).

Conclusion — When Does JEPA Planning Work vs Fail?

When it works

JEPA-based CEM planning is likely to succeed when:

1. **The task has slow, smooth dynamics** — where position-only prediction is sufficient and velocity doesn't dominate outcomes (e.g., navigation, slow manipulation).
2. **The embedding space is metrically aligned with the goal** — either by design (contrastive objectives) or because the task is simple enough that prediction accuracy alone induces alignment.
3. **The planning horizon is short relative to the dynamics timescale** — a 1–2 step prediction is much more accurate than a 10-step one for any model.
4. **The goal is reachable from many action sequences** — CEM does not require a precise gradient, only a cost landscape that is "roughly correct" so that elite samples cluster near the right direction.

Why it fails here

Billiards is a maximally hostile test case for JEPA planning:

Property	Why it's hard for JEPA
Collision dynamics	Sharp nonlinearity — one frame before/after contact looks nearly identical; the model must predict the exact post-collision velocity
Potting is rare	Only ~1 frame out of a typical 100-step episode is the potting frame; the model rarely learns this regime
Velocity is invisible	Single frames don't encode velocity; the model can't reason about momentum
Goal is a specific microstate	Reaching a pocket requires a precise shot; many trajectories are "close" in pixel space but physically fail
Long effective horizon	The cue must interact with the target which must interact with the pocket — a 3-step causal chain within 10 model steps

What the four experiments establish

1. **Embedding geometry alone cannot drive planning** (Experiments 1–2): Even with 8.5× better prediction accuracy and explicit embedding structure fixes, raw L2 distance in embedding space provides no useful gradient for CEM.
2. **A good cost function cannot rescue a bad dynamics model** (Experiment 4): The probe achieved $R^2=0.988$ for target position — near-perfect state decoding — yet planning still failed completely. The bottleneck was the predictor, not the cost.
3. **The real simulator solves it trivially** (Experiment 3): State-based CEM with the true dynamics pots the ball in 9–13 steps. The task is plannable; only the learned model is the obstacle.

The fundamental lesson: JEPA learns to predict *plausible* futures, not *physically accurate* futures. For tasks where approximate prediction is enough (e.g., "is there an obstacle ahead?"), JEPA planning works. For tasks where the outcome is determined by a precise physical event (a collision), the approximate model is worthless for planning regardless of cost function design.

Implementation Notes

- All experiments run on Apple Silicon M5 Max (MPS backend).
- Training: `precision="32", pin_memory=False, num_workers=0` required for MPS.
- Dataset actions are already normalised (mean $\approx$ 0, std $\approx$ 1 per dimension). CEM clamp of ± 3.0 covers $\sim 99.7\%$ of the action distribution.
- BilliardsEnv: `WINDOW_SIZE=512, RENDER_SIZE=96, FRICTION=0.96, MAX_STEPS=300, POCKET_RADIUS=20`, pockets at corners ($\sim 20, 20$), ($\sim 492, 20$), ($\sim 20, 492$), ($\sim 492, 492$).
- State vector: `[cue_x, cue_y, cue_vx, cue_vy, target_x, target_y, target_vx, target_vy, nearest_pocket_x, nearest_pocket_y]`.